C PROGRAMMING – ASSIGNMENT 2 (CO2)

ASSIGNMENT REPORT

Submitted by

SRIRAM

Register Number: 1911260194

Course: C Programming

Submission Date: 05/11/2026

Individual Assignment | Total Marks: 50

DECLARATION

I declare that this report is prepared for the individual C Programming Assignment 2 (CO2). The programs, calculations and explanations are presented for learning and verification. Before submission, I will compile and execute each program, capture my own output screenshots, record the actual measurements, and update the GitHub repository link and evidence.

This assignment examines the use of arithmetic, relational, logical, bitwise, assignment and conditional operators; operator precedence and associativity; decision-making statements; and looping constructs in C. The assignment contains seven questions and carries 50 marks. Questions 1–6 focus on applying and analysing programming concepts, while Question 7 requires a measured redesign of one selected program.

The report follows the sequence required by the assignment: predict, design, implement, measure, analyse, evaluate, diagnose, redesign and validate. The supplied assignment PDF is the source for the problem statements, expected outputs and submission requirements.

Learning objectives

Apply precedence and associativity rules to arithmetic expressions.

Demonstrate short-circuit evaluation and measure function-call counts.

Use bitwise operators to store and manipulate compact device states.

Compare if–else ladders, switch statements and lookup tables.

Implement prime checking with different loops and count divisions.

Design a loan repayment loop that terminates safely.

• Redesign a measured bottleneck and justify the trade-offs.

3. QUESTION 1 – PRECEDENCE, ASSOCIATIVITY AND THE DISCOUNT BUG (6 MARKS)

Problem: A pharmacy calculates price × quantity, applies a 15% senior-citizen discount, adds 5% GST, and then gives Rs. 50 off when the amount exceeds Rs. 1,000. The supplied expression is:

```
total = price * qty - price * qty * 15 / 100 * senior
        + price * qty * 5 / 100;
net = total > 1000 ? total - 50 : total;
count = count++ + 1;
```

3.1 Operator precedence and associativity

Multiplication, division and remainder have higher precedence than addition and subtraction. Operators at the same precedence level are evaluated left to right for these arithmetic operators. The relational comparison `>` is evaluated before the conditional operator `?:`. The conditional operator selects one of its two result expressions after its condition is evaluated.

The expression calculates the gross amount more than once. If `price`, `qty`, `senior` and `total` are integers, the percentage calculations use integer arithmetic and discard fractional parts. This can cause rounding down. The intended billing calculation should use a floating-point type and

explicit intermediate values.

3.2 Sample predictions

Price	Quantity	Senior?	Gross	15% discount	5% GST	Total after GST	Net after offer
500	3	Yes	1500.00	225.00	75.00	1350.00	1300.00
200	3	Yes	600.00	90.00	30.00	540.00	540.00
120	2	No	240.00	0.00	12.00	252.00	252.00

These are hand-calculated predictions using the intended business rule: discount first, then add GST to the discounted amount, then apply the Rs. 50 offer if the post-GST total is greater than Rs. 1,000. They are not terminal measurements.

3.3 Corrected implementation

```
#include <stdio.h>

int main(void) {
    double price, gross, discount, taxable, gst, total, net;
    int qty, senior;

    printf("Enter unit price, quantity and senior status (1/0): ");
    if (scanf("%lf %d %d", &price, &qty, &senior) != 3) return 1;

    gross = price * qty;
    discount = senior ? gross * 0.15 : 0.0;
    taxable = gross - discount;
    gst = taxable * 0.05;
    total = taxable + gst;
    net = (total > 1000.0) ? total - 50.0 : total;

    printf("Gross = %.2f\nDiscount = %.2f\nGST = %.2f\n",
           gross, discount, gst);
    printf("Total = %.2f\nNet = %.2f\n", total, net);
    return 0;
}
```

Logic: Calculate gross amount once, calculate the discount based on senior status, subtract it to obtain the taxable amount, calculate GST on that amount, and finally apply the fixed offer. Intermediate variables make the business sequence explicit.

3.4 Undefined behaviour and design decision

The statement `count = count++ + 1;` modifies `count` through the post-increment and also modifies it through assignment without a sequencing relationship that makes the modifications safe. This results in undefined behaviour in C. A compiler may warn under `-Wall` (often with an unsequenced-operation diagnostic), but the warning is not guaranteed to appear in every compiler version. Use `count++;` or `count += 1;` instead.

Alternative using intermediate variables: The corrected program already uses `gross`, `discount`, `taxable`, `gst`, `total` and `net`. The number of arithmetic operations is still constant, so time complexity remains O(1). A few extra scalar variables use O(1) auxiliary space. The improvement is readability, testability and reduced risk of precedence or integer-division errors.

Validation to record: Test a senior and non-senior bill, a total exactly equal to Rs. 1,000, and a total just above Rs. 1,000. Record actual console output and compare it with the hand

predictions.

4. QUESTION 2 – SHORT-CIRCUIT EVALUATION (4 MARKS)

Problem: A concession is granted when the passenger is a senior citizen and the ticket class is eligible, or the passenger has a verified disability card. The verification function is slow.

4.1 Logical condition

```
if ((senior && eligible) || verify_disability_card(id)) {
    printf("Concession granted\n");
}
```

For `&&`, if the left operand is false, the right operand is not evaluated. For `||`, if the left operand is true, the right operand is not evaluated. Therefore the verification call is skipped when `(senior && eligible)` is already true.

4.2 Function-call count experiment

Assume a reproducible sample of 1,000 passengers: 400 are both senior and eligible; 600 do not satisfy both conditions. With the expression above, the slow verification function is called only for the 600 passengers whose first combined condition is false. Thus the predicted call count is 600.

Implementation	Calls to slow verification (for this sample)	Reason
(senior && eligible) || verify()	600	Skip verification for 400 already-qualified passengers.
verify() || (senior && eligible)	1,000	Verification is evaluated first for every passenger.
(senior & eligible) | verify()	1,000	Bitwise operators evaluate both operands; no short circuit.

The sample proportions are an explicit experiment assumption, not measured passenger data. For an actual run, use a fixed 1,000-row test dataset and increment a global counter inside `verify_disability_card()`.

4.3 Example measurement code

```
static unsigned long verify_calls = 0;

int verify_disability_card(int id) {
    verify_calls++;
    /* Replace with a test stub or real verification. */
    return (id % 10 == 0);
}

/* Test order A */
if ((senior && eligible) || verify_disability_card(id)) {
    /* concession */
}
```

A nested-if implementation can preserve short-circuit behaviour if written carefully. Using `&` and `|` on integer conditions evaluates both operands and can cause unnecessary server calls. If the verification function now writes every checked passenger to an audit log, condition order becomes a correctness and audit-policy decision, not only a speed optimisation. If every passenger must be audited, call verification explicitly for every passenger and then evaluate the

concession rule.

Complexity: Across n passengers, the worst-case verification count is O(n). With short-circuiting, the actual count depends on the number of passengers failing the first condition. Auxiliary space is O(1), excluding the audit log. Reordering conditions is beneficial when it avoids expensive checks and does not change required side effects.

5. QUESTION 3 – BITWISE DEVICE-STATUS REGISTER (6 MARKS)

Problem: Store the ON/OFF state of up to eight devices in one unsigned byte. A bit value of 1 means ON and 0 means OFF. Device indices are LIGHT=0, FAN=1, AC=2, HEATER=3 and PUMP=4.

5.1 Completed bitwise functions

```
#include <stdio.h>
enum { LIGHT = 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4 };

unsigned char switch_on(unsigned char reg, int dev) {
    return (unsigned char)(reg | (1u << dev));
}
unsigned char switch_off(unsigned char reg, int dev) {
    return (unsigned char)(reg & ~(1u << dev));
}
unsigned char toggle(unsigned char reg, int dev) {
    return (unsigned char)(reg ^ (1u << dev));
}
int is_on(unsigned char reg, int dev) {
    return (reg & (1u << dev)) != 0;
}
void show(unsigned char reg) {
    for (int bit = 7; bit >= 0; bit--)
        putchar(((reg >> bit) & 1u) ? '1' : '0');
    putchar('\n');
}
int main(void) {
    unsigned char reg = 0;
    reg = switch_on(reg, LIGHT);
    reg = switch_on(reg, AC);
    show(reg);
    reg = toggle(reg, FAN);
    reg = switch_off(reg, LIGHT);
    show(reg);
    printf("AC is %s, PUMP is %s\n",
           is_on(reg, AC) ? "ON" : "OFF",
           is_on(reg, PUMP) ? "ON" : "OFF");
    return 0;
}
```

5.2 Register tracing

Operation	Binary register	Hex value
Initial value	00000000	0x00
Switch LIGHT on (bit 0)	00000001	0x01
Switch AC on (bit 2)	00000101	0x05
Toggle FAN (bit 1)	00000111	0x07

Operation	Binary register	Hex value
Switch LIGHT off (bit 0)	00000110	0x06

Expected output:

```
00000101
00000110
AC is ON, PUMP is OFF
```

5.3 Precedence, signedness and alternative storage

The expression `reg & 1u << dev` is parsed as `reg & (1u << dev)` because shift has higher precedence than bitwise AND. In contrast, `(reg & 1u) << dev` first tests only bit zero and then shifts that result. They are not equivalent. Use an unsigned type for the register and unsigned constants for masks; shifting into the sign bit of a signed integer can create implementation-sensitive or undefined behaviour depending on the operation.

An alternative is `unsigned char state[8]`, with one element per device. Individual state access is O(1) in either design. The byte register uses one byte per home; the array uses eight bytes per home. For 1,000,000 homes, the nominal storage is 1,000,000 bytes versus 8,000,000 bytes, excluding any alignment or container overhead. Counting ON devices in the bit register can inspect eight bits (constant work); counting an array scans eight elements, also constant for exactly eight devices. The bit register is more compact, while the array is easier to extend and inspect.

6. QUESTION 4 – DECISION-MAKING: ELSE-IF LADDER AND SWITCH (6 MARKS)

Problem: Convert marks to grades: 90–100 S, 80–89 A, 70–79 B, 60–69 C, 50–59 D, below 50 F, and invalid marks as '?'.

6.1 Else-if implementation

```
char grade_if(int marks) {
    if (marks < 0 || marks > 100) return '?';
    if (marks >= 90) return 'S';
    else if (marks >= 80) return 'A';
    else if (marks >= 70) return 'B';
    else if (marks >= 60) return 'C';
    else if (marks >= 50) return 'D';
    else return 'F';
}
```

6.2 Completed switch implementation

```
char grade(int marks) {
    if (marks < 0 || marks > 100) return '?';
    switch (marks / 10) {
        case 10:
        case 9: return 'S';
        case 8: return 'A';
        case 7: return 'B';
        case 6: return 'C';
        case 5: return 'D';
        default: return 'F';
    }
}
```

The initial range check is essential: C integer division truncates toward zero, so invalid negative marks should be rejected before using `marks / 10`. The switch groups case 10 and case 9 for grade S. Each return exits the function and prevents fall-through.

6.3 Boundary test table

Marks	Expected grade	Reason
100	S	Upper valid boundary
90	S	S lower boundary
89	A	Immediately below 90
80	A	A lower boundary
75	B	Middle of B band
60	C	C lower boundary
50	D	D lower boundary
49	F	Immediately below 50
0	F	Lower valid boundary
-1	?	Invalid
101	?	Invalid

A missing `break` in a switch allows execution to continue into the next case until a break, return or end of switch. `switch` compares one integral expression with constant case labels; it cannot directly express a range such as 75–84. An else-if ladder is direct for ranges, while switch is concise after mapping marks to tens.

Lookup-table alternative: `char grades[11] = {'F','F','F','F','F','D','C','B','A','S','S'};` and return `grades[marks / 10]` after validating marks. Lookup is O(1) time and uses O(1) space (11 characters). An else-if ladder may perform up to k comparisons, O(k), where k is the number of grade bands. A lookup table is fast and compact for fixed bands but less self-explanatory when rules become more complex. Scholarship rules involving grade and income can be handled with a switch on grade and a nested if for income, or a table if the mapping is stable.

7. QUESTION 5 – PRIME CHECKING AND LOOP COUNTS (7 MARKS)

Problem: The basic algorithm tests every divisor from 2 to n−1. The improved algorithm tests only divisors up to the square root because any larger factor must have a matching smaller factor.

7.1 Completed code using the square-root boundary

```
#include <stdio.h>
unsigned long checks = 0;

int is_prime(unsigned long n) {
    checks = 0;
    if (n < 2) return 0;
```

```
    for (unsigned long i = 2; i * i <= n; i++) {
        checks++;
        if (n % i == 0) return 0;
    }
    return 1;
}

int main(void) {
    unsigned long tickets[] = {97, 100, 7919, 1000003};
    for (int k = 0; k < 4; k++) {
        int p = is_prime(tickets[k]);
        printf("%8lu %-9s divisions = %lu\n",
               tickets[k], p ? "prime" : "not prime", checks);
    }
    return 0;
}
```

7.2 Expected division counts

Number	Result	Divisors tested	Division count
97	Prime	2 through 9	8
100	Not prime	2 (divisor found immediately)	1
7919	Prime	2 through 88	87
1000003	Prime	2 through 1000	999

The expected output in the assignment is 8, 1, 87 and 999 divisions respectively. The counter increments once for each modulo/divisibility test. Note that `i * i <= n` can overflow for sufficiently large unsigned values; a safer loop condition is `i <= n / i`, which avoids multiplication overflow.

7.3 Loop alternatives and boundary cases

The improved condition can be implemented using for, while or do-while. A do-while executes its body at least once, so it needs explicit handling for n < 2 and careful stopping for n=2 and n=3. For a do-while version, ensure that no invalid divisor is tested and that the loop condition is checked after the body.

Changing the condition to `i < n` returns to trial division up to n−1 and can dramatically increase the count. The Sieve of Eratosthenes is an alternative when many queries up to a known maximum N are expected: preprocessing takes O(N log log N), each later query is O(1), and the boolean array requires O(N) space. Trial division takes O(√n) time per query and O(1) auxiliary space.

8. QUESTION 6 – LOAN REPAYMENT AND TERMINATION (8 MARKS)

Problem: Each month, interest is added to the outstanding loan balance and a fixed repayment is made. The schedule must report monthly balance, number of months and total interest.

8.1 Loop design

A while loop is suitable because the number of months is not known in advance and the loop should continue while a positive balance remains. Use a safety limit such as 360 months. Do not use exact floating-point equality to determine completion; instead stop when the balance is less than or equal to a small tolerance or when the balance is reduced to zero after the final payment.

8.2 Sample loan schedule

Assumptions: principal = Rs. 10,000; monthly interest rate = 2%; repayment = Rs. 2,000. Interest is added first, then the repayment is subtracted. The final repayment is capped at the amount owed.

Month	Opening balance (Rs.)	Interest 2% (Rs.)	Payment (Rs.)	Closing balance (Rs.)
1	10,000.00	200.00	2,000.00	8,200.00
2	8,200.00	164.00	2,000.00	6,364.00
3	6,364.00	127.28	2,000.00	4,491.28
4	4,491.28	89.83	2,000.00	2,581.11
5	2,581.11	51.62	2,000.00	632.73
6	632.73	12.65	645.38	0.00

Approximate total interest = Rs. 645.38. The sixth payment is only the outstanding amount after interest, not the full Rs. 2,000.

8.3 Safe implementation

```
#include <stdio.h>

int main(void) {
    double balance = 10000.0, rate = 0.02, repayment = 2000.0;
    double total_interest = 0.0;
    int month = 0, limit = 360;

    if (repayment <= balance * rate) {
        printf("Repayment is not greater than monthly interest; loan will not reduce.\n");
        return 0;
    }

    while (balance > 0.005 && month < limit) {
        double interest = balance * rate;
        double due = balance + interest;
        double payment = (repayment < due) ? repayment : due;
        balance = due - payment;
        total_interest += interest;
        month++;

        if (balance < 0.005) balance = 0.0;
        printf("%d %.2f %.2f %.2f\n", month, interest, payment, balance);
    }

    printf("Months = %d\nTotal interest = %.2f\n", month, total_interest);
    if (balance > 0.0)
        printf("Safety limit reached; outstanding balance remains.\n");
    return 0;
}
```

If the repayment is less than or equal to monthly interest, the balance does not decrease and the loop may never terminate. The program therefore checks this condition before entering the loop.

A 360-month safety limit protects against unexpectedly long schedules.

8.4 Formula alternative and complexity

The assignment gives a direct logarithmic formula for estimating the number of months. A formula-based method can run in O(1) time and O(1) space, while monthly simulation takes O(m) time for m months and O(1) extra space. The formula is sensitive to the exact timing convention (interest before or after payment), floating-point precision and domain restrictions of logarithms. Monthly simulation is easier to audit because it produces the full repayment schedule; the formula is useful for a quick estimate.

9. QUESTION 7 – DAY-4 DESIGN CHALLENGE (13 MARKS)

Selected program: Q5 prime checking. The original improved version checks every integer divisor from 2 through √n. For large prime inputs, it performs the maximum number of modulo operations in that range.

9.1 Diagnose the bottleneck

For n = 1,000,003, the square-root method tests 999 candidate divisors (2 through 1000). Most of the work is spent testing even divisors after 2, even though every even number greater than 2 is known to be composite. This is avoidable work.

9.2 Measurable design objective

Reduce the number of divisibility tests for odd prime inputs by testing divisor 2 once and then testing only odd divisors from 3 upward, while preserving correct results for all integers n. The target is approximately half as many tests for large odd primes.

9.3 Redesigned pseudocode

```
function is_prime_optimized(n):
    if n < 2: return false
    if n == 2: return true
    if n is even: return false
    divisor = 3
    while divisor <= n / divisor:
        count one divisibility test
        if n % divisor == 0: return false
        divisor = divisor + 2
    return true
```

9.4 Redesigned C implementation

```
int is_prime_odd_only(unsigned long n, unsigned long *checks) {
    *checks = 0;
    if (n < 2) return 0;
    if (n == 2) return 1;
    if (n % 2 == 0) {
        (*checks)++;
        return 0;
    }

    for (unsigned long i = 3; i <= n / i; i += 2) {
        (*checks)++;
        if (n % i == 0) return 0;
    }
```

```
    return 1;
}
```

9.5 Predicted comparison

Input	Original √n checks	Odd-only checks (predicted)	Expected reduction
97	8	4	50%
100	1	1 (even test)	No increase
7919	87	43	About 50.6%
1000003	999	499	About 50.1%

For 1,000,003, the odd-only loop checks 3, 5, 7, …, 999: 499 candidates. It therefore reduces the count from 999 to 499, a reduction of 500 tests (about 50.1%). For 97, it checks 3, 5, 7, 9: four candidates after the separate handling of 2. The exact counter convention should be stated consistently: the table counts the even check for 100 as one divisibility test.

9.6 Complexity, memory and trade-offs

Both trial-division implementations remain O(√n) time in Big-O notation, because skipping even divisors changes the constant factor rather than the asymptotic growth rate. Both use O(1) auxiliary space. The redesigned method is expected to perform about half as many modulo tests for large odd prime inputs. The original method is simpler to read; the redesigned method is faster for large inputs but requires explicit handling of 2 and even numbers.

9.7 Validation plan

Compile both versions with the same compiler and optimisation flags. Run them on the same inputs: 2, 3, 4, 97, 100, 7919 and 1,000,003. Record prime/composite result, modulo count and elapsed time if measured. Counts are the primary comparison because elapsed time for small programs can be affected by system noise. Replace the predicted values above with actual execution counts if the implementation's counter includes the initial even check.

Result status: The comparison above is a mathematically derived prediction from the algorithms, not a claim of a completed terminal experiment. Insert the actual output screenshots and measured counts after executing both programs.

10. OVERALL OBSERVATIONS AND CONCLUSION

The assignment demonstrates that correct C programming requires more than writing syntactically valid statements. Precedence, integer arithmetic, evaluation order, side effects, bit representation, boundary conditions and loop termination all affect correctness. Explicit intermediate variables improve the clarity of financial calculations. Short-circuit evaluation avoids unnecessary function calls when side effects do not require every check. Bitwise masks provide compact storage for device states. Switch statements are useful for discrete values, while if–else ladders are more suitable for ranges. Square-root trial division and odd-divisor skipping reduce unnecessary work in prime checking. A safe loan loop must handle the final partial payment, non-amortising repayments and a maximum iteration limit.

The Day-4 redesign selects Q5 because its bottleneck can be measured directly through modulo-operation counts. Skipping even divisors reduces the predicted work for large odd primes by approximately half without changing the O(√n) asymptotic complexity or requiring additional memory. The final design should be accepted only after both versions are compiled, run with identical test data, and their actual counts are recorded.

Source basis: C Programming Assignment 2 – CO2, 10-page assignment handout supplied with this request. The handout specifies the problem statements, expected outputs, complexity topics, evidence requirements and marking scheme.

End of Report